

GEM: Graph-based Indexing for Efficient Multi-Vector Retrieval

Yao Tian^{1†}, Zhoujin Tian^{1†}, Xi Zhao¹, Ruiyuan Zhang², Xiaofang Zhou¹

¹The Hong Kong University of Science and Technology, China

²Hong Kong Generative AI Research & Development Center, China
{ytianbc, ztianaf, xzhaoca, zxf}@cse.ust.hk, zry@hkgai.org

ABSTRACT

Multi-vector retrieval, which encodes queries and data as sets of embeddings, has become the new paradigm in semantic search due to its superior query quality over traditional single-vector approaches. However, this accuracy gain comes with scalability challenges. Existing methods either sacrifice the rich set-level structure for speed, or retain it at the cost of inefficient search. To address this dilemma, we propose GEM, a novel framework that preserves the richness of set representations while leveraging the powerful pruning capabilities of a graph index. GEM builds a dual proximity graph architecture at the set level, comprising intra-cluster and global graphs, to support efficient cross-region traversal, with an adaptive TF-IDF-guided cluster assignment to reduce redundancy while preserving semantic coverage. To ensure stable navigation, GEM decouples the graph construction metric from the final scoring function and introduces semantic shortcuts to guide the search further. A centroid-based quantized distance dramatically accelerates both graph construction and traversal. By employing a multi-entry search strategy with cluster-guided early pruning, GEM ultimately achieves fast and accurate retrieval over the index. Extensive experiments on in-domain, out-of-domain, and multi-modal benchmarks demonstrate that GEM is up to 16x faster than state-of-the-art methods, while matching or surpassing their accuracy, offering a scalable solution to the multi-vector retrieval problem.

PVLDB Reference Format:

Yao Tian, Zhoujin Tian, Xi Zhao, Ruiyuan Zhang, and Xiaofang Zhou.
GEM: Graph-based Indexing for Efficient Multi-Vector Retrieval. PVLDB,
14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at
<https://github.com/eitbar/VecSetSearch>.

1 INTRODUCTION

The advent of Large Language Models (LLMs) has profoundly reshaped the landscape of information retrieval in recent years. By encoding diverse data into high-dimensional vectors, LLMs have

made vector representations the dominant data format, with efficient approximate nearest neighbor (ANN) search becoming the backbone of modern retrieval-augmented generation (RAG) systems [11, 23, 24, 39]. While this paradigm advanced from lexical to conceptual matching, flattening rich semantics into a single, holistic embedding vector inevitably leads to the loss of fine-grained details, causing irrelevant results and hallucinations in downstream LLMs [52]. To overcome this limitation, modern information systems are increasingly representing complex objects as sets of vectors [14, 20, 21, 27, 35, 43, 53]. Intuitively, the multi-vector representation can preserve more semantic nuance that single-vector representations often miss, enabling more precise matching. The success of this paradigm is exemplified by models like ColBERT [20], which generates a set of contextualized embeddings for each token in the query and document. The relevance score is then evaluated using Chamfer similarity (*a.k.a.* MaxSim operator, see Definition 1), which aggregates the maximum similarity of each query embedding to any document embedding. Compared to traditional single-vector retrieval, this new paradigm has demonstrated significant improvements in retrieval quality [18, 44, 45, 50], interpretability [7, 51], and generalization to out-of-domain queries [31, 54] across various IR benchmarks, making it the new frontier in semantic search.

However, these gains come at a steep cost in storage, computational overhead, and the complexity of scalable indexing and retrieval. Intuitively, storing multiple vectors instead of one inflates storage requirements by several orders of magnitude, and pairwise distance calculations across all vectors in the set-to-set comparison incur significantly higher costs. These issues can be alleviated by pruning less informative vectors or adapting techniques from single-vector search, such as ultra-low bit quantization, with minimal accuracy loss. However, the more formidable challenge lies in efficient and accurate indexing and retrieval mechanisms. Since the similarity measure involves a non-linear aggregation, multi-vector search cannot be directly handled by traditional, highly-optimized ANN indexes designed for single-vector cases. To address this, the community has explored several strategies. One straightforward idea is to adapt existing ANN indexes by pooling all vectors from the sets into a single index, regardless of the set-level structure [20, 37, 42, 43]. However, this bloats the index and generates many false positives for refinement. Another strategy [5] projects the vector set into a single vector through plausible distance-preserving mappings, enabling the use of existing ANN indexes. While this ensures efficiency, it sacrifices the fine-grained accuracy that multi-vector representations promise. An alternative [6] is to compress the set into a sketch, such as through locality-sensitive hashing (LSH) [3, 4]. However, without effective sketch-level pruning, the speedup is limited, and storage pressure increases due to the large number of hash values. To sum up, the notable capabilities of multi-vector

[†]Equal Contribution

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

search, coupled with the limitations of current methods in both efficiency and accuracy, highlight a clear research gap. This points to the need for novel indexing and retrieval framework specifically designed to handle multi-vector retrieval at scale.

To this end, we propose GEM, a novel **G**raph-based framework tailored for **E**fficient **M**ulti-Vector retrieval. Rather than a straightforward application of mature single-vector ANN methods, GEM is a series of co-designed optimizations for the unique challenges in multi-vector retrieval. Its core idea is to construct a proximity graph at the set level, preserving the semantic richness of vector sets while leveraging the powerful pruning capabilities of a graph index. To achieve this, GEM introduces a dual-graph structure, consisting of intra-cluster graphs and a global graph, where each vertex represents a vector set. The intra-cluster graphs ensure compact local structures, while sets shared across clusters naturally form bridges, creating a globally connected graph that supports efficient cross-region traversal and pruning. Paired with a learned TF-IDF-style cluster assignment scheme that links each set to only its most salient clusters, the graph minimizes index redundancy while maintaining semantic coverage. To address the non-metric nature of Chamfer distance, GEM decouples the construction metric from the final scoring function. It builds the graph using Earth Mover’s Distance (EMD) [41], a metric that ensures a stable graph topology. As EMD is an upper bound for the final Chamfer distance score, navigating the EMD-based graph is a reliable heuristic for finding high-quality Chamfer neighbors. To mitigate the prohibitive computational cost of EMD and Chamfer, GEM creates centroid-based quantization, converting the costly set-to-set distance calculations into efficient, codebook-based table lookups, significantly speeding up both index construction and querying. To further bridge the gap between these two measures, shortcuts are injected to directly connect vertices that are semantically related but may be far apart, accelerating search and helping escape local optima. Finally, a tailored, cluster-guided, multi-entry search algorithm upon our index structure is developed. The search starts by multiple relevant clusters and expands in parallel, while a cluster-guided pruning mechanism prevents the search from drifting into irrelevant regions of the graph, effectively balancing speed and accuracy. The main contributions of this paper are summarized as follows:

- We conduct a comprehensive analysis of existing multi-vector retrieval methods and a naive adaptation of graph indexes from the single-vector domain, identifying the key causes of their suboptimal performance.
- We propose GEM, a novel set-level graph framework that unifies the semantic richness of multi-vector representations with the efficiency of graph-based pruning, by a dual-graph design and metric decoupling.
- We introduce a series of co-designed architectural and algorithmic innovations, including an adaptive TF-IDF cluster assignment, semantic shortcuts, a centroid-based distance quantization, and a multi-entry search strategy with cluster-guided early pruning, to achieve an optimal accuracy-efficiency trade-off.
- Extensive experiments on in-domain, out-of-domain, and multi-modal benchmarks show that GEM significantly outperforms the state-of-the-art methods in both retrieval quality and latency, with a comparable or even smaller index size and indexing time.

The rest of the paper is organized as follows. Section 3 introduces the problem definition, analyzes the limitations of existing methods, and discusses a naive graph-based solution. The GEM framework is detailed in Section 4, followed by experimental results in Section 5. Section 2 reviews related works, and we conclude in Section 6.

2 RELATED WORK

2.1 Single-Vector Search

The ANN search problem has been extensively studied in the database community for decades [46]. However, nearly all well-known methods make the single-vector representation assumption, *i.e.*, both queries and data are encoded as single high-dimensional vectors in the same Euclidean space. Prominent approaches include LSH-based methods [15, 22, 29, 30, 47, 48, 56], quantization-based methods [9, 10, 12, 17], and graph-based methods [8, 16, 34, 36, 49, 55]. LSH-based methods are well-known for the strong theoretical guarantees on recall. Quantization-based methods significantly reduce memory usage and accelerate distance calculations. Recently, graph-based methods, which construct a proximity graph among vectors and find neighbors via greedy traversal, have become the de facto state-of-the-art, demonstrating superior performance in both accuracy and efficiency. However, these techniques are not directly applicable to the multi-vector setting. A detailed analysis of this gap is presented in Section 3.

2.2 Multi-Vector Search

Multi-vector search is a relatively new paradigm that has seen rapid adoption across various domains [19, 20, 26, 38] due to its high effectiveness. It was first introduced for document retrieval by ColBERT [20], which represents queries and documents as sets of token-level embeddings and employs a *late interaction* architecture to achieve strong performance. Subsequent work further improved the effectiveness of this paradigm, for instance by learning different weights for vectors of varying importance [14], or expanded it to other domains such as visual-language retrieval tasks [28, 53]. However, the reliance of these early works on a basic inverted list to index all individual embeddings created critical barriers to large-scale deployment. To improve scalability, ColBERTv2 [43] introduced a centroid-based compression scheme, where each token embedding is stored using the ID of its closest centroid and a low-bit residual (1–2 bits per component), significantly reducing storage overhead. However, the search process remained costly. PLAID [42] builds on this by further exploiting the centroids to aggressively prune irrelevant documents via a *centroid interaction* mechanism, substantially accelerating retrieval. More recently, EMVB [37] enhanced PLAID with system-level optimizations, including bit-vector filtering and SIMD-accelerated scoring. Alternative indexing strategies have also been explored. MUVERA [5] introduces fixed-dimensional encodings (FDEs) that approximate multi-vector similarity, enabling the use of optimized single-vector ANN indexes. While efficient, this method inevitably loses token-level alignment, risking reduced recall. DESSERT [6] proposed compressing vector sets into compact sketches, but its speedup is often limited by a lack of effective set-level pruning. Orthogonal to these indexing and search optimizations, XTR [21] modifies the training process itself, improving retrieval efficiency by simplifying the scoring mechanism.

CITADEL [25], on the other hand, reduces computation by selectively considering only a subset of tokens for relevance estimation, at the cost of retrieval quality.

3 PRELIMINARY

In this section, we formulate the multi-vector retrieval problem, analyze the limitations of existing methods, and present a naive solution. Frequently used notations are summarized in Table 1.

3.1 Problem Definition

DEFINITION 1 (CHAMFER DISTANCE [2]). *Given two sets of vectors $A, B \subset \mathbb{R}^d$, the Chamfer distance, a.k.a. the MaxSim operator, is defined as*

$$CH(A, B) = \sum_{a \in A} \min_{b \in B} d_X(a, b), \quad (1)$$

where d_X is the underlying distance measure, such as L2 distance.

REMARK 1. Besides L2, cosine distance is also commonly used, where max replaces min in the formulation. This case can be easily reformulated into a min problem by negating the similarity score. For consistency with traditional nearest neighbor search conventions, we adopt the min formulation in this work.

PROBLEM STATEMENT (MULTI-VECTOR RETRIEVAL PROBLEM). *Given a database $\mathcal{D} = \{P_1, P_2, \dots, P_N\}$, where each $P_i = \{p_1, p_2, \dots, p_{m_i}\}$ is a set of vectors with each $p_j \in \mathbb{R}^d$, and a query $Q = \{q_1, q_2, \dots, q_{m_q}\}$ with each $q_j \in \mathbb{R}^d$, the multi-vector retrieval problem, a.k.a. the vector set search problem, returns the top- k sets in $\mathcal{D}$ that are most similar to Q in terms of Chamfer distance, namely:*

$$\mathcal{R} = k\text{-arg min}\{CH(Q, P) | P \in \mathcal{D}\}. \quad (2)$$

REMARK 2. Multi-vector retrieval has been widely adopted in various domains and systems [19, 20, 26, 38]. A prominent example is ColBERT [20] for document retrieval, where queries and documents are encoded into sets of token-level vectors. By convention, we use the terms "document" and "vector set" interchangeably, as well as "token" and "vector" interchangeably, throughout this work. However, it is worth noting that multi-vector retrieval itself is a general problem, agnostic to the data's origin. A "vector" could represent an image patch, a chunk of text, or other forms of data.

3.2 Why Not Use Existing Methods

Existing multi-vector retrieval techniques typically fall into three categories, each with fundamental limitations in efficiency, scalability, or accuracy.

3.2.1 Keyword-style Search. These methods [20, 37, 42, 43] treat each vector in a set independently and pool them all into a single large ANN index, e.g., an inverted list. At query time, each query vector retrieves its top- k nearest neighbors, and the results are merged and re-ranked using Chamfer similarity. This approach is straightforward but suffers from efficiency issues. Vector-level matches do not always imply set-level relevance, leading to a large number of irrelevant candidates for further refinement. For example, a document about "rocky beaches in Portugal" may be retrieved for the query "family-friendly beaches in Europe" due to a shared term like "beaches" despite the clear mismatched intent. Additionally,

Table 1: List of key notations

Notation	Description
$\mathcal{D}$	The database of vector sets
N	Number of vector sets in the database
P, Q	A vector set from the database, a query vector set
p, q	Token vectors in d -dimensional space
m	Number of token vectors in a set
d	Dimensionality of each vector
C_{quant}	Fine-grained centroids
C_{index}	Coarse-grained centroids
$NN(p)$	Nearest centroid assigned to vector p
$C_{\text{top}}(P)$	Clusters assigned to set P after TF-IDF pruning

searching an index with all individual vectors from each document is inefficient, e.g., 10k documents with 100 vectors each lead to a million-level ANN task. As a result, this method struggles to scale with either large datasets or sets containing many vectors.

3.2.2 Sketch-Based Set Approximation. This core idea of these methods [6] is to compress each vector set into a compact sketch, e.g., using LSH, to approximate multi-vector similarity. At query time, hashed representations are compared to retrieve the top- k most similar candidates, which are then re-ranked using Chamfer similarity. While this method avoids pairwise vector interactions, it is bottlenecked by an exhaustive search over the sketches. As typically $m \ll N$, it offers limited speedup without an efficient set-level pruning mechanism. Moreover, storing multiple hashes per set increases space usage, which limits scalability on large datasets.

3.2.3 Single-Vector Approximation. The core idea behind these methods [5] is to map each vector set into a single vector, such that $\text{dist}(F(Q), F'(P)) \approx CH(Q, P)$, thereby enabling the usage of off-the-shelf ANN indexes, such as HNSW [34]. This simplification, however, leads to an accuracy issue. A theoretically sound approximation requires prohibitively high dimensionality of the projection space that even well-optimized ANN indexes struggle to handle efficiently. More fundamentally, this approach contradicts the motivation for multi-vector representations to preserve fine-grained detail and begs the question of why not directly train a single-vector embedding model from the outset.

3.3 A Naive Graph-Based Solution

Based on the above investigation, we find a fundamental dilemma among the existing methods: they either sacrifice accuracy by aggressively compressing vector sets (as in single-vector or sketch-based approaches) or sacrifice efficiency by lacking an effective set-level pruning mechanism (as in keyword-style or sketch-based approaches). This dilemma motivates us to explore a new direction. Inspired by the success of graph-based indexes in single-vector search, we pose a question: what if we build an approximate proximity graph (APG) at the set level? A naive solution is to construct an APG $G(V, E)$, where each vertex $v \in V$ represents a vector set from the dataset $\mathcal{D}$ and an edge in E connects two sets that are sufficiently similar based on Chamfer distance. The graph construction

and search processes are the same as those in single-vector graph-based methods, and thus we refrain from repeating them here. In principle, such a graph would address the aforementioned dilemma. By treating each vector set as a vertex, it preserves the structural and semantic richness of multi-vector representations, ensuring high retrieval accuracy. Meanwhile, the graph structure itself provides a powerful pruning mechanism, enabling fast navigation to the most promising candidates. However, this direct extension fails due to two critical issues:

- (1) Fragmented neighbors and local optima. Chamfer distance is not a metric and lacks triangle inequality, which undermines core assumptions behind graph-based search. For example, even if Q is close to P_1 and P_1 is close to P_2 , Q may still be far from P_2 . As a result, the nearest neighbors of Q are scattered across the graph, making them difficult to reach via local hops and slowing down convergence. Without strong connectivity among these neighbors, greedy search paths become oscillatory and unstable, often getting trapped in local optima—terminating early even when better candidates exist several hops away, ultimately degrading recall.
- (2) Lack of cheap distance approximation and pruning. While graph structures provide a powerful pruning framework, the exorbitant cost of computing the Chamfer distance remains a critical bottleneck. It is impractical to perform set-to-set comparisons between the query and all candidate neighbors at each step of the search traversal. A practical solution, therefore, necessitates a cheaper distance approximation, effective pruning strategies, and more informed entry points.

We will empirically validate these analyses in Section 5, where this naive approach, named MVG, is included as a baseline.

4 GEM

To address the limitations of existing approaches and the challenges exposed by the naive graph-based solution, we propose GEM — a novel Graph-based indexing framework for Efficient Multi-vector retrieval. While the naive graph solution performs poorly, we believe graph-based structures hold strong potential for accurate and efficient vector search. To unlock this potential, GEM introduces a series of optimizations across four dimensions: graph structure, pruning strategy, search algorithm, and quantization design. The key innovations are as follows:

- (1) We propose a set-level clustering scheme, where each document is assigned to only a small number of semantically relevant clusters, guided by adaptive TF-IDF-style weighting over centroids. This reduces index redundancy while preserving semantic coverage (Section 4.1 and Section 4.4.2).
- (2) We design a novel dual graph structure, featuring both intra-cluster graphs and a unified global graph naturally bridged by sets shared between clusters. This design guarantees global connectivity while preserving local structure (Section 4.2 and Section 4.3).
- (3) We decouple the similarity measures used for graph construction and search: Earth Mover’s Distance (EMD) is used for graph construction due to its metric properties, while Chamfer distance is retained for search. Since EMD upper bounds Chamfer, vertices that are close under EMD are also close under

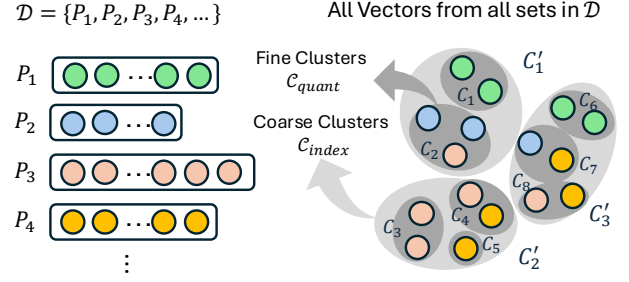


Figure 1: Two-Stage Clustering

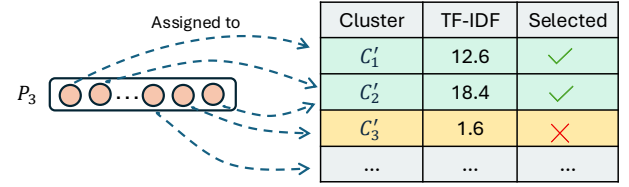


Figure 2: Cluster Assignment with TF-IDF

Chamfer, ensuring reasonable navigation. We further augment the graph with shortcuts, directly linking semantically close pairs under Chamfer but may require several hops under EMD, which improves search efficiency and helps escape local optima (Section 4.2 and Section 4.4.1).

- (4) We introduce quantized versions of EMD and Chamfer to alleviate the high cost of set-to-set similarity computation. By leveraging centroid-level codebooks and quantized representations derived from cluster centroids, we make both the indexing and search phases significantly more efficient (Section 4.2 and Section 4.5).
- (5) We devise a new cluster-guided, multi-entry search strategy. The search is initialized from multiple semantically relevant clusters, enabling parallel, layer-wise expansion over the global graph. A cluster-guided early pruning mechanism aggressively culls unpromising paths, preventing the search from drifting into irrelevant regions (Section 4.5).

4.1 Clustering for Set-Level Graph Indexing

4.1.1 Two-Stage Clustering. We adopt a two-stage clustering strategy to balance the needs of accurate distance approximation and efficient set-level indexing. This process is visually outlined in Figure 1. Specifically, we first perform K-means clustering over all vectors across all sets in $\mathcal{D}$ to obtain a set of fine-grained centroids $C_{\text{quant}} = \{C_1, C_2, \dots, C_{k_1}\}$. These centroids capture low-level semantic details and serve as a vocabulary for our subsequent quantized distance approximations, i.e., qEMD during graph construction as described in Section 4.2.2 and qCH during search in Section 4.5.3. In the second stage, we cluster C_{quant} into a smaller set of coarser centroids $C_{\text{index}} = \{C'_1, C'_2, \dots, C'_{k_2}\}$, which defines a cluster space for set-level indexing. As illustrated in Figure 1, all vectors are first grouped into 8 fine-grained clusters, which are then organized into 3 coarse clusters that serve as higher-level semantic bins. Then, we assign each vector $p \in P$ to its nearest cluster centroid in C_{index}

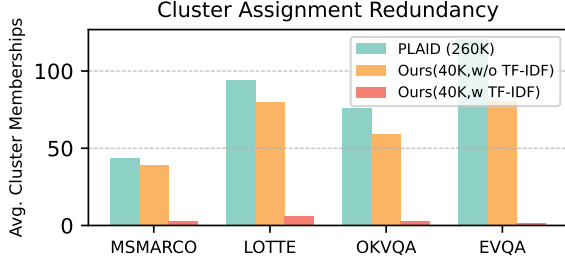


Figure 3: Average # of Clusters Each Set is Assigned to

according to $L2$ distance:

$$\text{NN}(p) = \arg \min_{C'_i \in C_{\text{index}}} \|p - C'_i\|_2. \quad (3)$$

As a result, a set $P = \{p_1, p_2, \dots, p_m\}$ is approximately represented as a multiset of centroids:

$$C(P) = \{\text{NN}(p_1), \text{NN}(p_2), \dots, \text{NN}(p_m)\}. \quad (4)$$

To obtain a set-level clustering, a naive way is to associate a set with every cluster if one or more of its tokens are assigned to that centroid. For example, P_3 is linked to all three clusters C'_1, C'_2, C'_3 . However, this can cause a vector set to appear in an excessive number of clusters, especially when it contains uninformative tokens, such as "is", "the", and "of". This leads to index redundancy and increased query-time overhead. To mitigate this issue, we employ a TF-IDF-guided pruning strategy, as described below.

4.1.2 TF-IDF Guided Cluster Pruning. The intuition is straightforward: focus only on clusters associated with important or distinctive tokens, while ignoring uninformative ones. This way, we reduce redundancy while preserving the semantic richness of multi-vector representations. To achieve this, we compute a TF-IDF-style importance score over the centroid vocabulary C_{index} to reflect how salient the cluster is to the set P . Specifically, we define term frequency as:

$$\text{TF}(C'_j, P) = |\{p \in P : \text{NN}(p) = C'_j, C'_j \in C_{\text{index}}\}|, \quad (5)$$

which counts how many token vectors in P are associated with a cluster C_j . Then, we apply an inverse document frequency to downweight clusters shared across many sets:

$$\text{IDF}(C'_j) = \log \frac{N}{1 + |\{P' \in \mathcal{D} : \text{TF}(C'_j, P') > 0\}|}, \quad (6)$$

where N is the number of vector sets in the database. The overall TF-IDF score is then:

$$\text{Score}(C'_j, P) = \text{TF}(C'_j, P) \cdot \text{IDF}(C'_j). \quad (7)$$

We then construct a weighted centroid profile for each document:

$$\mathcal{S}(P) = [(C'_{j_1}, s_{j_1}), (C'_{j_2}, s_{j_2}), \dots, (C'_{j_m}, s_{j_m})], \quad (8)$$

where $s_{j_i} = \text{Score}(C'_{j_i}, P)$ indicates how strongly cluster C'_{j_i} represents the semantics of P . Finally, we only assign P to the top- r clusters with the highest TF-IDF scores:

$$C_{\text{top}}(P) = \text{Top-}r_{C'_j \in C(P)} \text{Score}(C'_j, P). \quad (9)$$

Clearly, there is no single r value that works best for all: some sets are well represented by a few clusters, while others may require broader cluster coverage. We defer the details of how to determine

Algorithm 1: Index Construction Pipeline

Input: Database $\mathcal{D}$, two-stage cluster counts k_1, k_2 , degree limit M , parameter f , training pairs $\mathcal{T}$,
Output: GEM proximity graph G

- 1 Initialize G with $V = \mathcal{D}$ and $E = \emptyset$;
- 2 $C_{\text{quant}}, C_{\text{index}}, C_{\text{top}} \leftarrow \text{CLUSTERANDASSIGN}(\mathcal{D}, k_1, k_2)$;
- 3 **foreach** $C_i \in C_{\text{index}}$ **do**
- 4 $G \leftarrow \text{CLUSTERGRAPHANDCROSSBRIDGES}(C_i, C_{\text{quant}}, C_{\text{top}}, f, M)$;
- 5 $G \leftarrow \text{INJECTSHORTCUTS}(G, \mathcal{T}, M)$;
- 6 **return** G

r adaptively to Section 4.4.2. Figure 2 provides a concrete example of this pruning. For P_3 , our TF-IDF guided mechanism identifies that its connection to cluster C'_3 is weak (score of 1.6). Therefore, only the two clusters with high semantic relevance (scores 18.4 and 12.6) are selected as its indexing targets. This pruning step ensures that each document is linked only to its most representative clusters, improving both graph construction efficiency and candidate generation efficiency.

DISCUSSION 1. PLAID adopts a similar clustering idea, but differs from our approach in two aspects. It performs only fine-grained token-level clustering and directly assigns each document to every cluster that any of its tokens maps to. However, we found this strategy less suitable for our graph-based indexing framework. In particular, PLAID’s large number of fine-grained clusters results in very few documents per cluster—often only dozens—making it unnecessary and inefficient to build a graph index per cluster. Moreover, the high redundancy in cluster memberships results in costly graph construction and many redundant edges during query processing. Therefore, to better support our graph-based design, we adopt a two-stage clustering strategy. The fine-grained clustering in the first stage is crucial for accurate distance approximation, while second-stage coarser clusters, combined with TF-IDF pruning, significantly reduce per-document cluster memberships, striking a better balance between granularity and efficiency.

Figure 3 compares the average number of clusters each document is assigned to under different strategies across four datasets. The PLAID bars correspond to PLAID’s default configuration with 260K fine-grained clusters, resulting in high redundancy. The Ours (40K, w/o TF-IDF) bars represent a coarser 40K-cluster setup without applying TF-IDF guided cluster filtering, while the Ours (40K, w/ TF-IDF) bars show the results after pruning. As shown, our approach dramatically reduces the number of cluster memberships, e.g., from 43.8 to 2.9 on MSMARCO (a 93.4% reduction), from 94 to 5.8 on LOTTE (93.8%), from 76 to 3 on OKVQA (96%), and from 120.8 to 1.7 on EVQA (98.6%), alleviating index redundancy while preserving semantic coverage.

4.2 Cluster-Level APG with qEMD

4.2.1 Stable Graph Construction with EMD. To construct a stable and navigable graph, we propose using EMD [41] as the edge weight metric. EMD is a well-established distance function that satisfies the triangle inequality, ensuring a well-structured graph topology with guaranteed monotonicity. A key insight supporting this metric decoupling is the bounding relationship between EMD and Chamfer

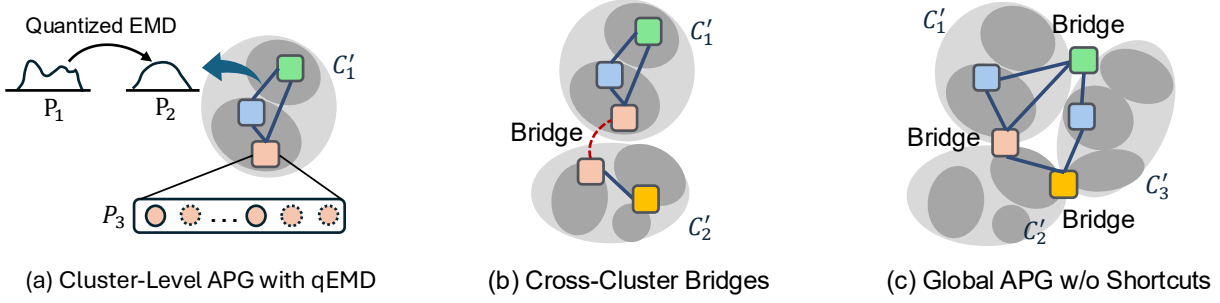


Figure 4: Overview of Global Graph Construction

distance. It can be proven that for any two vector sets Q and P :

$$\text{CH}(Q, P) \leq \text{EMD}(Q, P). \quad (10)$$

This inequality implies that if two sets are close under EMD, they are at least as close—if not closer—under Chamfer distance. Consider a search path that is currently at vertex P_1 , then the distance from P_1 to its neighbor P_2 has a predictable upper bound: $\text{CH}(Q, P_2) \leq \text{EMD}(Q, P_2) \leq \text{EMD}(Q, P_1) + \text{EMD}(P_1, P_2)$. Consequently, navigating toward the nearest neighbor under EMD is a reliable heuristic for finding a reasonable candidate in the Chamfer space. This mitigates oscillatory search behavior and the local optima trap. EMD is formally defined as follows: given two vector sets $P_1 = \{p_1, \dots, p_{m_1}\}$ and $P_2 = \{p'_1, \dots, p'_{m_2}\}$, and a transport plan $T = \{t_{ij}\}$, then

$$\text{EMD}(P_1, P_2) = \min_T \sum_{i=1}^{m_1} \sum_{j=1}^{m_2} t_{ij} \cdot d_X(p_i, p'_j) \quad (11)$$

subject to the following constraints:

$$\sum_{j=1}^{m_2} t_{ij} = \frac{1}{m_1}, \quad \sum_{i=1}^{m_1} t_{ij} = \frac{1}{m_2}, \quad t_{ij} \geq 0 \quad (12)$$

where d_X is L2 distance. While EMD offers desirable properties for graph construction, its computation is prohibitively expensive [1, 3, 13, 32, 33]. To make EMD feasible for large-scale index construction, we introduce an efficient quantized approximation.

4.2.2 Quantized EMD via Clustering. We reduce the computational cost of EMD by leveraging cluster centroids C_{quant} as described in Section 4.1. This enables us to estimate EMD via centroid-level distances instead of raw vector comparisons:

$$\tilde{d}_X(p_i, p'_j) = d_X(\text{NN}(p_i), \text{NN}(p'_j)) \quad (13)$$

where $\text{NN}(p_i)$ and $\text{NN}(p'_j)$ are the nearest centroids in C_{quant} to tokens p_i and p'_j , respectively. All centroid-to-centroid distances are precomputed and stored in a codebook of size $k_1 \times k_1$, enabling fast lookups. Moreover, the smaller centroid space leads to much sparser transport plans, which further speed up EMD computation. Formally, the quantized EMD (qEMD) is as follows:

$$\text{qEMD}(P_1, P_2) = \min_T \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} t_{ij} \cdot d_X(\text{NN}(p_i), \text{NN}(p'_j)) \quad (14)$$

The graph is incrementally constructed following the pipeline in Algorithm 1. For each cluster C_i , we invoke **CLUSTERGRAPHANDCROSSBRIDGES** (Algorithm 2) to build a cluster-level graph while

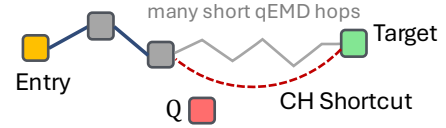


Figure 5: Graph Enhancement via Shortcuts

Algorithm 2: Build Cluster Graph with Cross Bridges

Input: Cluster C_i , current global graph $G = (V, E)$, $C_{\text{top}}(\cdot)$, f , M
Output: Updated global graph G

```

1 foreach  $P \in C_i$  do
2    $C \leftarrow f$  ANN results of  $P$  in the current graph of  $C_i$  by qEMD;
3   if  $E[P] = \emptyset$  then
4     foreach  $P' \in C$  do
5       Add a bidirectional edge  $(P, P')$  to  $E$ ;
6       if  $\text{degree of } P' > M$  then
7         Remove the least similar edge from  $E[P']$ ;
8   else
9      $G \leftarrow \text{UPDATEBRIDGES}(P, C, G, C_{\text{top}}(P), M)$ ; // Alg. 3
10 return  $G$ 

```

handling cross-cluster bridges. As shown in Algorithm 2, for each vector set $P \in C_i$, we find its top- f ANNs by performing the search algorithm (Algorithm 5) over the current cluster graph, starting from an arbitrary entry point in C_i and using qEMD as the distance metric (Line 2). If P is new to the current graph G (Line 3), we directly connect it to its neighbors (Line 5) and ensure that each neighbor P' does not exceed the degree limit M by removing the least similar edges if necessary (Line 7). Otherwise, P has already been inserted in previous clusters $\{C_i \mid i < j, C_i \in C_{\text{top}}(P)\}$, which means P can serve as a cross-cluster bridge. In this case, we invoke **UPDATEBRIDGES** (Algorithm 3) to merge the old and new neighbor candidates while preserving inter-cluster and intra-cluster connectivity (Line 9), as detailed in the next section.

4.3 Global Graph via Cross-Cluster Bridges

This section details how the local graph structures are woven into a single, globally connected graph. Creating a unified graph is critical for both search and storage efficiency, as it avoids a slow, cluster-by-cluster traversal, and prevents the redundancy of creating separate graph vertices for the same set appearing in multiple

Algorithm 3: Update Bridges

Input: Set P , new neighbors C_{new} , $G = (V, E)$, $C_{\text{top}}(P)$, M
Output: Updated global graph G

```
1  $C_{\text{old}} \leftarrow$  existing neighbors of  $P$  in  $E$ ;  
2  $C_{\text{all}} \leftarrow C_{\text{new}} \cup C_{\text{old}}$ ;  
3 if  $|C_{\text{all}}| \leq M$  then  
4    $C_{\text{final}} \leftarrow C_{\text{all}}$ ;  
5 else  
6    $C_{\text{final}} \leftarrow M$  closest vertices of  $P$  in  $C_{\text{all}}$  by qEMD;  
7   foreach  $C_i \in C_{\text{top}}(P)$  do  
8     if  $C_{\text{final}} \cap C_i = \emptyset$  then  
9        $S_i \leftarrow \{P \in C_{\text{all}} \mid P \in C_i\}$ ;  
10      Replace farthest in  $C_{\text{final}}$  with closest node from  $S_i$ ;  
11 Replace  $E[P]$  with edges  $(P, P')$  for all  $P' \in C_{\text{final}}$ ;  
12 return  $G$ 
```

clusters. As shown in Algorithm 3, for a bridge set P , we gather both the neighbors already connected to P from earlier clusters and the new neighbors identified in the current cluster (Line 1-2). If the total number of neighbors is within the degree limit M , we retain them all (Line 4). Otherwise, the set is pruned to the top- M closest neighbors (Line 6). To ensure P preserves its role as bridges, we enforce that at least one neighbor from each cluster in $C_{\text{top}}(P)$ is included in the final neighbor set (Line 7-10). For example, if P belongs to clusters C'_2 and C'_4 , this step ensures its final neighbor list will contain at least one node from C'_2 and one from C'_4 . This prevents newly added connections from overwriting prior inter-cluster links and preserves global connectivity. Finally, the neighborhood of P is updated to reflect the new connection set (Line 11). Note that each P corresponds to a *unique* vertex in the global graph: logically assigned to multiple clusters, but physically stored as a single shared node. This design maintains intra-cluster structural coherence, supports efficient cross-cluster traversal via inter-cluster bridges, and eliminates redundant vertex duplication. Figure 4 further illustrates the graph construction process. In Figure 4(a), P_3 is assigned to cluster C'_1 based on its informative vectors (solid-edge circles). Within the cluster, a local graph is constructed using qEMD-based similarity. Since P_3 also belongs to another cluster C'_2 , it becomes a natural bridge between clusters, as shown in Figure 4(b). Figure 4(c) presents a complete toy example of the resulting dual-graph structure.

4.4 Graph Enhancement

4.4.1 Inject Shortcuts. We observed that some queries may require many short hops to reach a relevant result. This issue arises because, while EMD provides an upper bound on Chamfer distance, the reverse does not hold. As a result, two vector sets may be close under Chamfer but remain distant in EMD space, preventing direct connections during graph construction and leading to longer traversal paths at query time. To mitigate this, we propose a lightweight, supervised graph augmentation technique that injects semantic shortcuts into the graph index. A shortcut is a direct edge connecting two vertices that are labeled as semantically close but might be distant in the graph. As detailed in Algorithm 4, we leverage

Algorithm 4: Inject Shortcuts

Input: Graph $G = (V, E)$, training pairs $\mathcal{T}$, M
Output: Augmented GEM graph G

```
1 foreach  $(Q, P) \in \mathcal{T}$  do  
2    $C \leftarrow f'$  ANN results for  $Q$  in  $G$ ;  
3   if  $P \notin C$  and both  $P_{\text{top}}, P$  have degree  $\leq M$  then  
4      $P_{\text{top}} \leftarrow$  1-ANN in  $C$ ;  
5     Add a undirected edge  $(P_{\text{top}}, P)$  to  $E$ ;  
6 return  $G$ 
```

training pairs (Q, P) , where P is the labeled ground-truth containing the necessary information to answer the query Q , to augment the graph. For each pair, we run the search algorithm (Algorithm 5) over the current graph to find the query’s top- f' ANNs (Line 2). If P is not among them and both P_{top} and P have remaining degree capacity, we add an edge between them (Line 3-5). Since test queries are generally assumed to follow a similar distribution as the training data, such edges act as a shortcut, allowing future queries to reach the correct region more directly. As illustrated in Figure 5, the shortcut helps the search bypass long paths and sub-optimal results. By adding EMD-based structural connectivity with Chamfer-based semantic alignment, we enhance graph reachability, search efficiency, and accuracy.

4.4.2 Learned Cluster Cutoff. As discussed in Section 4.1.2, a fixed cut-off threshold r is suboptimal. To address this, we introduce an adaptive technique that employs a lightweight decision tree classifier [40] to predict an optimal per-set r . Our key idea is retrieval-oriented: retain the minimum cluster representation required for search-time discoverability. Specifically, we first generate r_{label} for each training pair (Q, P) . The label is defined as the rank of the first cluster in the TF-IDF-sorted profile $\mathcal{S}(P)$ that intersects with Q ’s relevant cluster set $C_{\text{query}}(Q)$. If no intersection occurs within the top- r_{max} (default 10), the label is set to r_{max} . Then, the decision tree is trained with the top- r_{max} TF-IDF scores (padded if necessary) and the number of vectors in P as input, and outputs the number of clusters r to retain. During the indexing pipeline, the trained model infers per-set r , which is then used for final cluster assignments. This strategy effectively reduces the number of redundant cluster assignments while preserving the necessary semantic coverage for effective retrieval.

4.5 Query Processing

Building on our index structure, we propose a cluster-guided beam search algorithm tailored for efficient multi-vector retrieval. The complete search procedure is detailed in Algorithm 5 and Figure 6 provides a concrete example of the query process. At a high level, the algorithm improves upon traditional single-vector greedy search from three complementary perspectives: (1) coarse-grained cluster filtering, (2) informative multi-entry point initialization, and (3) cluster-aware early pruning during graph traversals.

4.5.1 Cluster Filtering. In multi-vector retrieval, distance computations are significantly more expensive than in the single-vector setting. To reduce this cost, it is crucial to eliminate highly unlikely vector sets as early as possible—ideally before any graph

Algorithm 5: Cluster-Guided Beam Search

Input: Query Q , G , C_{quant} , C_{index} , parameters t, ef, k

Output: k ANNs to Q

// Cluster Filtering (Section 4.5.1)

```

1 Compute relevance scores  $S_{c,q} = C \cdot Q^T$ ;
2  $C_{\text{query}} \leftarrow$  union of top- $t$  centroids for each  $q_i \in Q$ ;
  // Multi-Entry Point initialization (Section 4.5.2)
3  $E \leftarrow$  one random node from each  $C_j \in C_{\text{query}}$ ;
  // Parallel Beam Search (Section 4.5.3)
4 Initialize a shared result heap  $\mathcal{R} \leftarrow E$ , a shared visited set  $\mathcal{V} \leftarrow E$ ;
5 Initialize a local queue  $\mathcal{W}_{ep}$  for each  $ep \in E$ ;
6 while any  $\mathcal{W}_{ep}$  is not empty do
7   foreach non-empty  $\mathcal{W}_{ep}$  in parallel do
8      $P \leftarrow$  pop the element in  $\mathcal{W}_{ep}$  closest to  $Q$  by qCH;
9      $\tau \leftarrow$  current furthest distance in  $\mathcal{R}$ ;
10    if  $\text{qCH}(Q, P) > \tau$  then
11      mark  $\mathcal{W}_{ep}$  as empty;
12      continue;
13    foreach  $P' \in \text{Neighbors}(P)$  do
14      if  $P' \notin \mathcal{V}$  and  $C_{\text{top}}(P') \cap C_{\text{query}} \neq \emptyset$  then
15        compute  $\text{qCH}(Q, P')$ ;
16        insert  $P'$  into  $\mathcal{W}_{ep}$ ,  $\mathcal{V}$ , and  $\mathcal{R}$ ;
17        if  $|\mathcal{R}| > ef$  then
18          Remove the furthest from  $\mathcal{R}$ ;
19      update  $\tau$ ;
20  $\mathcal{R} \leftarrow$  rerank top- $k$  results in  $\mathcal{R}$  by CH;
21 return  $\mathcal{R}$ 

```

traversal begins. For this purpose, we draw inspiration from the initial filtering phase of ColBERTv2, which is extremely fast for coarse candidate selection. While this strategy alone may yield a large number of false positives, our downstream graph traversal module can refine these results with high precision. This rationale leads us to retain ColBERTv2’s initial filtering strategy to generate coarse-grained cluster candidates. Specifically, we compute a query-centroid relevance matrix:

$$S_{c,q} = C \cdot Q^T, \quad (15)$$

where $C \in \mathbb{R}^{k_2 \times d}$ is the centroid matrix and $Q \in \mathbb{R}^{m_q \times d}$ is the query token matrix. For each query token q_i , we select its top- t nearest centroids and take the union across all tokens to form a relevant cluster set $C_{\text{query}} \subseteq C$. This early pruning step is shown in Line 1-2 in Algorithm 5 and visually illustrated in Figure 6, where irrelevant clusters (in gray) are filtered out before the graph traversal.

4.5.2 Multi-Entry Point Initialization. After identifying the relevant clusters, we initialize the search from multiple entry points, one from each cluster in C_{query} (Line 3). As shown in Figure 6, two yellow vertices (ep1 and ep2) from two different clusters are randomly selected as the initial entry points. This multi-entry strategy, combined with the semantic shortcuts on the index side, effectively mitigates the fragmentation of Chamfer nearest neighbors and enhances overall retrieval effectiveness.

4.5.3 Cluster-Guided Parallel Beam Search. Given the selected entry points, we perform a cluster-guided, multi-path parallel search

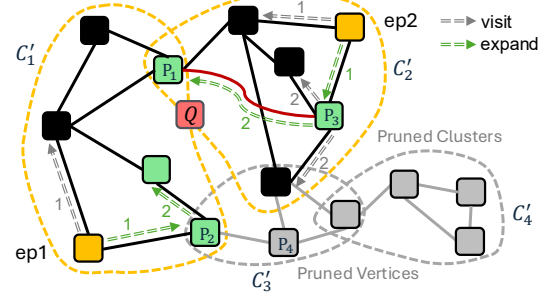


Figure 6: Search in GEM

over the graph G using quantized Chamfer distance (qCH). Similar to qEMD, qCH serves as a lightweight yet effective proxy for the expensive pairwise computation between vector sets, formalized as follows:

$$\text{qCH}(Q, P) = \sum_{q \in Q} \min_{p \in P} d_X(\text{NN}(q), \text{NN}(p)), \quad (16)$$

where $\text{NN}(\cdot)$ is the nearest centroids in C_{quant} . As elaborated before, although Chamfer is not used during index construction, it is upper-bounded by the EMD metric, leading that EMD-near neighbors are also reasonable candidates under Chamfer. Line 4-21 in Algorithm 5 details the query process. First, we initialize a global result heap $\mathcal{R}$, a global visited set $\mathcal{V}$, and a local priority queue $\mathcal{W}_{ep}$ for each entry point $ep \in E$ (Line 4-5). The search then proceeds in parallel from these entry points. Within each thread, we repeatedly pop the element P with the lowest qCH to the query (Line 8) until $\mathcal{W}_{ep}$ is empty (Line 7). We denote τ as the current furthest distance to Q in $\mathcal{R}$. If $\text{qCH}(Q, P)$ is already worse than τ , the current path is deemed unpromising and terminated (Line 9-12). Otherwise, we continue expanding P ’s neighbors. To boost query efficiency, we apply a cluster-aware pruning condition (Line 14). That is, if P' would pull the search into an irrelevant cluster (one not in C_{query}), it is directly filtered out. If the P' passes this filter, the qCH score is computed, and the local queue $\mathcal{W}_{ep}$, result heap $\mathcal{R}$, and visited set $\mathcal{V}$ are updated accordingly (Line 15-16). $\mathcal{R}$ maintains a maximum of ef candidates, evicting the farthest node when necessary (Line 17-18). Note that both $\mathcal{R}$ and $\mathcal{V}$ are globally shared across threads, enabling implicit communication. If one path discovers a closer node, it updates the τ (Line 19), which allows other paths to prune their search space earlier. Similarly, once a vertex is visited by any thread, it is skipped by all others, preventing redundant computation. The search terminates when all $\mathcal{W}_{ep}$ queues are exhausted. In the final step, the top- k candidates in $\mathcal{R}$ are re-ranked using exact Chamfer distance before being returned.

EXAMPLE 1. Figure 6 gives an example of the GEM search process for 1-ANN with $ef = 2$. P_1 is the ground-truth nearest neighbor. Assume that the cluster filtering stage identifies two relevant clusters, $C_{\text{query}} = \{C'_1, C'_2\}$, while pruning C'_3 and C'_4 . The search then begins in parallel from two entry points: ep1 and ep2 (yellow squares). In the first iteration (labeled with 1), the path of ep1 expands to P_2 , while ep2 discovers P_3 . These two candidates are inserted into the shared result heap $\mathcal{R}$. In the second iteration (labeled with 2), the path from ep1 continues by expanding from P_2 . When considering its neighbors, P_4 is

Table 2: Summary of Datasets

Datasets	# Corpus	# Vectors	# Queries	Modality
MS MARCO v1	8.8M	597M	6980	Text-only
LoTTe pooled	2.4M	339M	2931	Text-only
OK-VQA	114K	14M	5046	Text + Image
EVQA	50K	9.7M	3750	Text + Image

immediately pruned without a distance calculation because it resides in a pruned cluster. Concurrently, the other path expanding from P_3 jumps directly to P_1 via a semantic shortcut (highlighted in red), bypassing what would otherwise have been a multi-hop traversal.

5 EXPERIMENTS

In this section, we conduct extensive experiments on real-world datasets to evaluate our GEM. We implement GEM¹ in C++ compiled with g++ using -Ofast optimization and openMP for parallelism. All experiments are run on a Ubuntu server with 4 Intel(R) Xeon(R) Gold 6218 CPUs (160 threads) and 1.5 TB RAM.

5.1 Experimental Settings

5.1.1 Datasets. We evaluate GEM’s performance on four widely-used multi-vector search benchmarks, which are detailed in Table 2. MS MARCO v1 is used for in-domain evaluation with a retriever trained specifically for this task, while LoTTe serves as an out-of-domain benchmark. To further evaluate performance in complex multi-modal settings, we include two vision-language retrieval datasets, OK-VQA and EVQA, both involving queries with textual and visual content. For MS MARCO and LoTTe, we follow prior work and employ ColBERTv2 [43] to encode queries and documents into sets of vectors, each vector with a dimensionality of $d = 128$. For multi-modal datasets, we adopt preFLMR [28], a fine-grained late-interaction retriever that jointly encodes text and image content into multi-vector representations ($d = 128$). Both ColBERTv2 and preFLMR adopt the late interaction paradigm with set-to-set similarity computed via Chamfer distance, and have shown strong performance in text-only and multi-modal retrieval, respectively.

5.1.2 Competitors. We compare GEM with three representative methods in multi-vector retrieval, as mentioned in Section 2: a keyword-style approach PLAID [42], a sketch-based set approximation method DESSERT [6], and a single-vector approximation technique MUVERA [5]. We also include a graph-based baseline as discussed in Section 3.3, denoted as the multi-vector graph (MVG). To ensure basic competitiveness, here we use qCH for both indexing and search in our experiments.

5.1.3 Parameter Settings. All competitors are tuned to their best performance on each dataset using their publicly available codes or settings in the original papers. By default, all methods perform top-100 ANN search. For PLAID, we set the number of centroids to 262K and compress all vectors to 2 bits per dimension for MSMARCO, LoTTe; 8 bits and 32K for OKVQA, EVQA. For DESSERT, we set $C = 7$, $L = 64$ for MSMARCO, LoTTe and set $C = 4$, $L = 768$ for OKVQA, EVQA. For MUVERA, we adopt $R_{reps} = 20$, $K_{sim} = 5$,

and $D_{proj} = 32$. For our GEM, the number of clusters is scaled with dataset size to balance granularity and indexing cost. We set $|C_{quant}| = 262K$ for MSMARCO and LoTTe, and 32K for OK-VQA and EVQA. $|C_{index}| = 40K$ for MSMARCO, 10K for LoTTe, and 1K for OK-VQA and EVQA. Index construction uses $M = 24$ and $ef_construct = 80$. At query time, we set $t = 4$ and vary ef_search to trade off efficiency and accuracy. For MVG, we adopt the same indexing parameters as GEM. We build on the widely-established C++ library hnsplib, extending it to better support multi-vector retrieval.

5.1.4 Evaluation Metric. We adopt four metrics to evaluate query performance: Recall, Mean Reciprocal Rank (MRR), Success, and query latency; and two metrics to evaluate indexing performance: indexing time and index size. Given a query set Q , with ground-truth set $\mathcal{G}_Q$ for each query $Q \in Q$, and top- k retrieved results $\mathcal{R}_Q^{(k)}$, Recall@ k measures the proportion of relevant items retrieved:

$$R@k = \frac{1}{|Q|} \sum_{Q \in Q} \frac{|\mathcal{G}_Q \cap \mathcal{R}_Q^{(k)}|}{|\mathcal{G}_Q|}. \quad (17)$$

It is worth noting that our definition of recall here evaluates end-to-end task performance, as the ground truth $\mathcal{G}_Q$ is results truly containing the answer to a query. This differs from traditional ANN benchmarks where the ground truth is exact k NN obtained by brute-force search in the embedding space. Our metric is therefore more challenging and practical. MRR@ k captures the reciprocal rank of the first relevant item, and Success@ k indicates whether any relevant item is present in the top- k , defined as follows:

$$MRR@k = \frac{1}{|Q|} \sum_{Q \in Q} \frac{1}{\text{rank}_Q}, \quad (18)$$

$$S@k = \frac{1}{|Q|} \sum_{Q \in Q} \mathbb{I}[\mathcal{G}_Q \cap \mathcal{R}_Q^{(k)} \neq \emptyset]. \quad (19)$$

Query latency is reported as the average runtime over 10 repeated runs for each dataset.

5.2 Performance Analysis

5.2.1 Query Performance. In this section, we study the end-to-end retrieval performance of all methods under default settings, as summarized in Table 3, Table 4, and Figure 7.

Table 3 provides an overview of recall, success, MRR, and end-to-end latency for all methods across all datasets. Clearly, GEM offers the best overall query performance on all datasets. It adapts well to in-domain, out-of-domain, and more complex multi-modal datasets, outperforming all competitors on every query quality metric while maintaining the lowest latency. In particular, GEM achieves speedups of up to 2.5× on MSMARCO, 2.2× on LoTTe, 3.9× on OKVQA, and 2.8× on EVQA, all with better query quality. This experimentally validates our design goal: preserving the semantic richness of multi-vector representations to achieve high accuracy, while leveraging an efficient graph index for speed. The reason GEM achieves the best performance can be concluded as follows: 1) Compared with PLAID, GEM indexes at a set-level granularity, treating the entire vector set as the basic unit. This is fundamentally different from PLAID, which index all vectors individually. By doing so, GEM significantly reduces the number of false positive candidates returned due to token-level matches, avoiding the

¹<https://github.com/eitbar/VecSetSearch>

Table 3: End-to-end Retrieval Performance Overview

Method	MSMARCO				LOTTE				OKVQA				EVQA			
	R@100	S@100	MRR@10	T (ms)	R@100	S@100	MRR@10	T (ms)	R@100	S@100	MRR@10	T (ms)	R@100	S@100	MRR@10	T (ms)
PLAID	0.913	0.919	0.395	352	0.745	0.910	0.558	462	0.400	0.735	0.225	570	0.895	0.895	0.476	211
DESSERT	0.890	0.905	0.372	344	0.748	0.893	0.541	362	0.371	0.723	0.163	642	0.837	0.837	0.370	552
MUVERA	0.902	0.915	0.385	310	0.736	0.901	0.547	441	0.369	0.723	0.219	605	0.878	0.878	0.471	446
MVG	0.895	0.905	0.421	360	0.701	0.879	0.555	231	0.381	0.719	0.230	427	0.880	0.880	0.468	450
GEM (Ours)	0.915	0.920	0.447	140	0.749	0.910	0.592	210	0.410	0.754	0.236	165	0.895	0.895	0.477	197

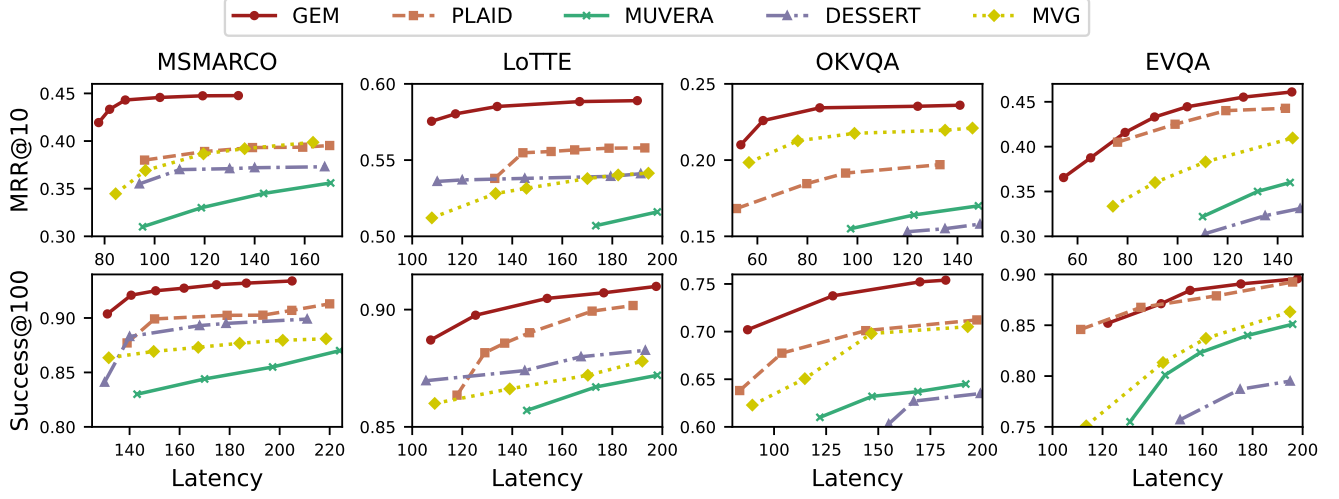


Figure 7: Accuracy-Latency Trade-off Overview

need for expensive refinement and boosting efficiency. 2) Compared with DESSERT, which is often bottlenecked by an exhaustive scan of all set sketches, GEM’s graph structure provides a powerful pruning mechanism, allowing GEM to navigate directly to the most promising candidates, resulting in significantly higher efficiency. 3) Compared with MUVERA, GEM avoids the significant accuracy degradation caused by compressing a vector set into a single vector. Instead, it preserves the set-level structure and semantic nuances, supported by a dedicated graph framework. This architectural choice allows GEM to achieve superior query quality easily, while MUVERA struggles to compete even when its FDEs are tuned to an impractically high 20480 dimensions. 4) Compared with the naive MVG, while it preserves multi-vector inputs, its naive graph suffers from fragmented neighbors, local optima, and lacks effective distance approximation and pruning strategies.

Figure 7 provides a more comprehensive view of the trade-off between query accuracy and efficiency of all methods. The recall-latency curves are omitted as they follow almost the same trend as the success-latency curves. From the figures, we have the following observations: (1) GEM consistently achieves the best trade-off. Among all methods, GEM requires the least time to reach the same MRR or Success, indicating the most favorable balance between accuracy and efficiency. For the same MRR@10, GEM is up to 16x faster than DESSERT (on OKVQA), up to 10x faster than MUVERA (on OKVQA), and up to 5x faster than PLAID (on

MSMARCO). (2) Visual-language retrieval is inherently more challenging. In OKVQA and EVQA, even with increased search time, Success@100 reaches only about 75–89%, while in text-only QA tasks, our algorithm can easily exceed 90%. Notably, this bottleneck is not on the retrieval side: even brute-force search on OKVQA and EVQA achieves only around 77–90% Success@100. This points to the need for stronger embedding models and improved multimodal feature alignment mechanisms. (3) GEM and PLAID are highly robust, maintaining stable performance across all datasets. PLAID is often the second-best method. In contrast, DESSERT and MUVERA exhibit substantial performance variation across datasets.

Table 4 reports how query quality and latency change as the number of returned results k increases. Due to space limitations, we only present results for two representative scenarios. We make the following observations: (1) GEM consistently achieves the best query performance across all k values, offering higher Recall@ k and Success@ k while maintaining significantly shorter query times. For example, on MSMARCO, GEM requires only 88 ms, 140 ms, and 212 ms for $k = 10, 100$, and 1000 respectively, whereas the best baselines require 222 ms (PLAID), 310 ms (MUVERA), and 336 ms (DESSERT) to reach comparable recall. On LOTTE, GEM completes in 133 ms, 210 ms, and 251 ms, while the fastest baselines still require 288 ms (PLAID), 362 ms (DESSERT), and 461 ms (MUVERA) to achieve similar quality. (2) Our naive solution, MVG, is competitive with some well-tuned baselines for small k . However,

Table 4: End-to-End Retrieval Performance Varying k

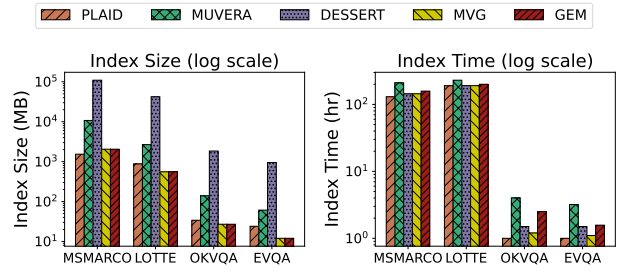
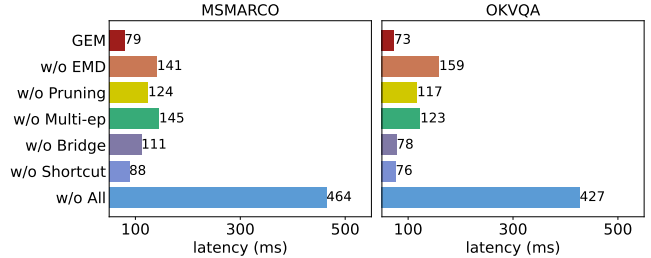
Method	MSMARCO			LoTTE		
	R@10/T	R@100/T	R@1k/T	S@10/T	S@100/T	S@1k/T
PLAID	68.8/222	91.3/352	97.5/352	76.8/288	91.0/462	94.5/462
DESSERT	66.2/298	89.0/344	97.2/336	74.7/362	89.3/362	94.3/477
MUVERA	67.7/286	90.2/310	97.1/444	74.1/397	90.1/441	94.4/461
MVG	69.2/293	89.5/360	94.2/436	74.7/303	87.9/331	94.2/442
GEM	71.1/88	91.5/140	97.5/212	77.3/133	91.0/210	94.5/251

on MSMARCO, its recall saturates at 94.2% when $k = 1000$. Even increasing `ef_search` and spending more time fails to recover the correct results. This limitation is due to the naive graph structure: Chamfer distance does not satisfy the triangle inequality, resulting in many local optima. Without GEM’s metric decoupling and additional strategies, such as shortcut connections and multi-entry search to explore from multiple regions, the naive approach struggles to handle these challenging queries. (3) As expected, in the out-of-domain setting, retrieval is more challenging than in-domain, and all methods require more time to achieve good quality. GEM remains consistently more accurate and faster, often requiring only 1/2 or even 1/3 of the time of other methods.

5.2.2 Indexing Performance. The indexing time and index size of all methods under default settings are summarized in Figure 8. We make two key observations: (1) GEM’s index size is small (e.g., 2 GB on MSMARCO, 27MB on OKVQA), significantly smaller than MUVERA (10.5 GB on MSMARCO, 139 MB on OKVQA) and DESSERT (107 GB on MSMARCO, 1.8GB on OKVQA). This compactness is attributed to the graph structure of GEM, which primarily stores edge information. This storage overhead is independent of whether the setting is under single- or multi-vector, and is negligible compared to the dataset size. In contrast, DESSERT requires storing a large number of hash values generated by LSH, while MUVERA needs to store Fixed Dimensional Encodings (FDEs) for all vector sets, which scale directly with the dataset size and make it less suited to large-scale scenarios. Although GEM’s index is slightly larger than PLAID (1.5 GB on MSMARCO, 34MB on OKVQA), GEM delivers a substantial advantage in both efficiency and accuracy. (2) On MSMARCO and LoTTE, all methods have comparable indexing time. This is because PLAID, DESSERT, and GEM all rely on a clustering step, which dominates the construction cost. MUVERA does not involve clustering but must convert every vector set into a single vector, which is also time-consuming. Both clustering and vector conversion can be accelerated on GPUs, making this step less of a practical concern. On smaller datasets, OKVQA and EVQA, the clustering cost becomes less significant, and GEM’s graph construction is slower than PLAID’s IVF and DESSERT’s LSH indexing. MUVERA, which requires both vector conversion and graph construction, remains the slowest. Overall, GEM achieves competitive indexing time and compact index size while providing superior retrieval performance.

5.3 Ablation Study

In this section, we evaluate how much each component of GEM contributes to the overall performance. We disable key modules

**Figure 8: Indexing Performance on All Datasets****Figure 9: Ablation Study on GEM’s Components.**

individually and compare the query latency required to achieve a comparable retrieval accuracy. As shown in Figure 9, removing different components results in varying degrees of latency increase.

5.3.1 w/o EMD distance. We replace the EMD approximation with qCH for graph construction, which doubles the latency on both datasets. The absence of metric properties degrades the graph quality, confirming the importance of the decoupling strategy in GEM.

5.3.2 w/o dynamic TF-IDF cluster pruning. We disable the adaptive adjustment of clusters per document and fix $|C_{top}| = 3$ for all documents. That is because a fixed threshold hurts recall for long, topically diverse documents, while short documents suffer efficiency loss from retaining unnecessary clusters. This result highlights the importance of dynamic TF-IDF-guided pruning in balancing accuracy and efficiency.

5.3.3 w/o multi-path strategy. We disable multi-path search from multiple entry points, instead enqueueing all entry points into a single search queue W and expanding greedily. This change increases latency by 70–80%. The slowdown occurs because, without synchronized paths sharing the visited and best-found result sets, pruning is less aggressive and the search space contracts more slowly. To reach the correct results, a larger `ef_search` is needed, which increases latency. These results confirm that multi-entry search accelerates convergence.

5.3.4 w/o bridge constraint. In this variant, when the degree of a bridge P exceeds the limit, we retain only its M closest neighbors, rather than enforcing it to keep at least one neighbor from each cluster in $C_{top}(P)$. This change greatly degrades performance, with the drop on MSMARCO more pronounced than on OKVQA due to

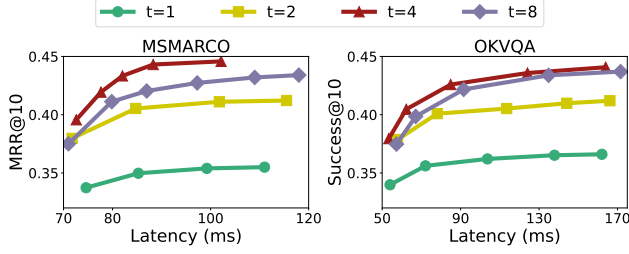


Figure 10: Impact of t on Retrieval Performance

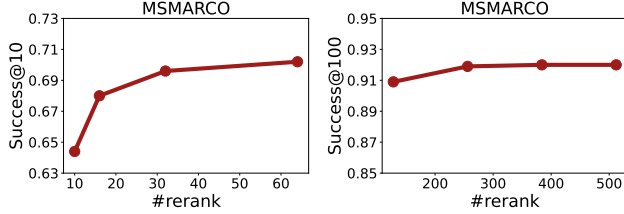


Figure 11: Impact of $\#rerank$ on Retrieval Performance

the larger corpus size. This result shows the importance of bridges in maintaining graph connectivity, especially on large datasets.

5.3.5 w/o shortcuts enhancement. We remove shortcuts that directly connect sets that may be close under Chamfer but require many hops to reach over the EMD graph. This leads to a modest degradation in query latency. This result indicates that while the co-designed architecture of GEM is already robust for most queries, the shortcut mechanism is a useful addition for handling a few hard-to-find cases.

It is worth noting that these components are co-designed to work synergistically. The optimal performance of GEM relies on their combined effect. Removing all of them (w/o all) leads to a severe degradation, with latency increasing by 5–6x.

5.4 Further Analysis

5.4.1 Effect of t . In this experiment, we study the impact of parameter t , which controls the number of nearest clusters each $q_i \in Q$ identifies in the early pruning step. We vary $t \in \{1, 2, 4, 8\}$ and report results on MSMARCO and OKVQA for brevity. Conceptually, increasing t makes the pruning more conservative: pruning fewer clusters to potentially increase recall, but at the cost of higher latency. As shown in Figure 10, both datasets exhibit similar trends. $t = 1$ proves to be too aggressive, leading to insufficient accuracy. Performance improves substantially at $t = 2$, while the gains begin to saturate as t increases further to 8. Therefore, we set $t = 4$ by default in our experiments as it provides a favorable balance between accuracy and efficiency.

5.4.2 Effect of $\#rerank$. In this experiment, we evaluate the impact of $rerank_k$. This parameter determines the size of the candidate pool retrieved from the graph search, which is then re-ranked using the exact Chamfer distance to obtain the final top- k results. Figure 11 reports results for $k = 10$ and 100. For $k = 10$, the choice of $rerank_k$ has a clear effect on accuracy: a larger $rerank_k$ increases

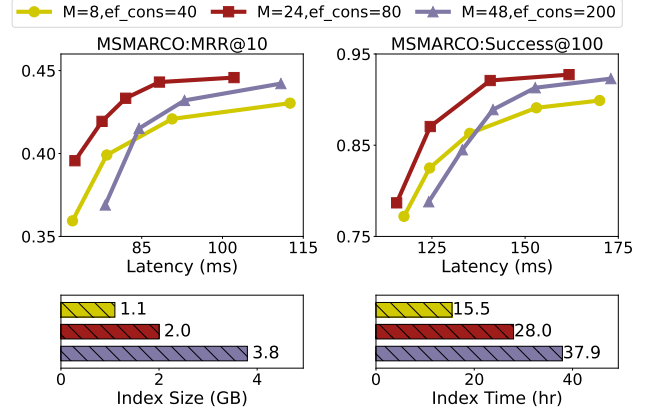


Figure 12: Impact of M and $ef_construction$

the likelihood that the true best results are included and ranked near the top. For $k = 100$, increasing $rerank_k$ has a more modest impact. This is because our algorithm is already highly effective, with a strong probability of placing the true results within its top-100 candidates, leaving limited room for further improvement.

5.4.3 Effect of Index Parameters. In this experiment, we investigate the impact of two key indexing parameters: the maximum number of neighbors per vertex (M) and the size of the candidate list during construction ($ef_construction$). As shown in Figure 12 on MSMARCO, with $M = 8$ and $ef_construction = 24$, the index is small and fast to build, but the graph quality is insufficient, resulting in lower query accuracy. Increasing to $M = 48$ and $ef_construction = 200$ significantly enlarges the index size and construction time, but query performance improves only marginally. This is because each vertex connects to too many neighbors, increasing the number of candidates to verify at each hop and thereby hurting latency. Our default configuration of $M = 24$ and $ef_construction = 80$ achieves the best overall trade-off, offering competitive indexing performance and superior query efficiency and accuracy.

6 CONCLUSION

This work presented GEM, a graph-based framework tailored for efficient multi-vector retrieval. GEM’s architecture features a set-level dual-graph index, metric decoupling, learned TF-IDF pruning, semantic shortcuts, quantized distance estimation, and a cluster-guided multi-path search. Experiments across diverse benchmarks show that GEM achieves superior accuracy and efficiency over state-of-the-art methods, with a comparable or even smaller index size and indexing time. These results highlight GEM’s potential as a foundation for deploying multi-vector search systems at scale.

For future work, we see ultra-low-bit quantization as a promising direction. One barrier to large-scale deployment of multi-vector representations is their substantially higher storage requirement compared to single-vector methods. Developing techniques that compress these vectors into highly compact formats with negligible accuracy loss will be critical for enabling production-scale adoption of multi-vector retrieval.

REFERENCES

- [1] Alexandr Andoni, Piotr Indyk, and Robert Krauthgamer. 2008. Earth mover distance over high-dimensional spaces. In *SODA*. SIAM, 343–352.
- [2] Ainesh Bakshi, Piotr Indyk, Rajesh Jayaram, Sandeep Silwal, and Erik Waingarten. 2023. Near-Linear Time Algorithm for the Chamfer Distance. In *NeurIPS*.
- [3] Moses Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *STOC*. ACM, 380–388.
- [4] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SCG*. ACM, 253–262.
- [5] Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. 2024. MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. *CoRR* abs/2405.19504 (2024).
- [6] Joshua Engels, Benjamin Coleman, Vihan Lakshman, and Anshumali Shrivastava. 2023. DESSERT: An Efficient Algorithm for Vector Set Search with Vector Set Queries. In *NeurIPS*.
- [7] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. A White Box Analysis of ColBERT. In *ECIR (2) (Lecture Notes in Computer Science)*, Vol. 12657. Springer, 257–263.
- [8] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.
- [9] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2025. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3 (2025), 202:1–202:26.
- [10] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 3 (2024), 167.
- [11] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2312.10997 (2023).
- [12] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Trans. Pattern Anal. Mach. Intell.* 36, 4 (2014), 744–755.
- [13] Kristen Grauman and Trevor Darrell. 2006. Approximate Correspondences in High Dimensions. In *NIPS*. MIT Press, 505–512.
- [14] Sebastian Hofstätter, Omar Khattab, Sophia Althammer, Mete Sertkan, and Alan Hanbury. 2022. Introducing Neural Bag of Whole-Words with ColBERT: Contextualized Late Interactions using Enhanced Reduction. In *CIKM*. ACM, 737–747.
- [15] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search. *PVLDB* 9, 1 (2015), 1–12.
- [16] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).
- [17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (2011), 117–128.
- [18] Omar Khattab, Christopher Potts, and Matei Zaharia. 2021. Relevance-guided Supervision for OpenQA with ColBERT. *Trans. Assoc. Comput. Linguistics* 9 (2021), 929–944.
- [19] Omar Khattab, Christopher Potts, and Matei A. Zaharia. 2021. Baleen: Robust Multi-Hop Reasoning at Scale via Condensed Retrieval. In *NeurIPS*. 27670–27682.
- [20] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *SIGIR*. ACM, 39–48.
- [21] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Zhao. 2023. Rethinking the Role of Token Retrieval in Multi-Vector Retrieval. In *NeurIPS*.
- [22] Yifan Lei, Qiang Huang, Mohan S. Kankanhalli, and Anthony K. H. Tung. 2020. Locality-Sensitive Hashing Scheme based on Longest Circular Co-Substring. In *SIGMOD Conference*. ACM, 2589–2599.
- [23] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- [24] Junnan Li, Dongxu Li, Silvio Savarese, and Steven C. H. Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *ICML (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, 19730–19742.
- [25] Minghan Li, Sheng-Chieh Lin, Barlas Oguz, Asish Ghoshal, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. CITADEL: Conditional Token Interaction via Dynamic Lexical Routing for Efficient and Effective Multi-Vector Retrieval. In *ACL (1)*. Association for Computational Linguistics, 11891–11907.
- [26] Yulong Li, Martin Franz, Md. Arafat Sultan, Bhavani Iyer, Young-Suk Lee, and Avirup Sil. 2022. Learning Cross-Lingual IR from an English Retriever. In *NAACL-HLT*. Association for Computational Linguistics, 4428–4436.
- [27] Weizhe Lin, Jinghong Chen, Jingbiao Mei, Alexandru Coca, and Bill Byrne. 2023. Fine-grained Late-interaction Multi-modal Retrieval for Retrieval Augmented Visual Question Answering. In *NeurIPS*.
- [28] Weizhe Lin, Jingbiao Mei, Jinghong Chen, and Bill Byrne. 2024. PreFLMR: Scaling Up Fine-Grained Late-Interaction Multi-modal Retrievers. In *ACL (1)*. Association for Computational Linguistics, 5294–5316.
- [29] Wanqi Liu, Hanchen Wang, Ying Zhang, Wei Wang, Lu Qin, and Xuemin Lin. 2021. EL-LSH: An early-termination driven I/O efficient incremental c-approximate nearest neighbor search. *VLDB J.* 30, 2 (2021), 215–235.
- [30] Kejing Lu, Hongya Wang, Wei Wang, and Mineichi Kudo. 2020. VHP: Approximate Nearest Neighbor Search via Virtual Hypersphere Partitioning. *Proc. VLDB Endow.* 13, 9 (2020), 1443–1455.
- [31] Simon Lupart, Thibault Formal, and Stéphane Clinchant. 2023. MS-Shift: An Analysis of MS MARCO Distribution Shifts on Neural Retrieval. In *ECIR (1) (Lecture Notes in Computer Science)*, Vol. 13980. Springer, 636–652.
- [32] Qin Lv, Moses Charikar, and Kai Li. 2004. Image similarity search with compact data structures. In *CIKM*. ACM, 208–217.
- [33] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2006. Ferret: a toolkit for content-based similarity search of feature-rich data. In *EuroSys*. ACM, 317–330.
- [34] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [35] Oscar R. Moll, Manuel Favela, Samuel Madden, Vijay Gadepally, and Michael J. Cafarella. 2023. SeeSaw: Interactive Ad-hoc Search Over Image Databases. *Proc. ACM Manag. Data* 1, 4 (2023), 260:1–260:26.
- [36] Javier Alvaro Vargas Muñoz, Marcos André Gonçalves, Zanon Dias, and Ricardo da Silva Torres. 2019. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search. *Pattern Recognit.* 96 (2019).
- [37] Franco Maria Nardini, Cosimo Rulli, and Rossano Venturini. 2024. Efficient Multi-vector Dense Retrieval with Bit Vectors. In *ECIR (2) (Lecture Notes in Computer Science)*, Vol. 14609. Springer, 3–17.
- [38] Ashwin Paranjape, Omar Khattab, Christopher Potts, Matei Zaharia, and Christopher D. Manning. 2022. Hindsight: Posterior-guided training of retrievers for improved open-ended generation. In *ICLR*. OpenReview.net.
- [39] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *CoRR* abs/2305.15334 (2023).
- [40] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [41] Yossi Rubner, Carlo Tomasi, and Leonidas J. Guibas. 2000. The Earth Mover’s Distance as a Metric for Image Retrieval. *Int. J. Comput. Vis.* 40, 2 (2000), 99–121.
- [42] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. PLAID: An Efficient Engine for Late Interaction Retrieval. In *CIKM*. ACM, 1747–1756.
- [43] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *NAACL-HLT*. Association for Computational Linguistics, 3715–3734.
- [44] Katherine Thai, Yapei Chang, Kalpesh Krishna, and Mohit Iyyer. 2022. RELiC: Retrieving Evidence for Literary Claims. In *ACL (1)*. Association for Computational Linguistics, 7500–7518.
- [45] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *NeurIPS Datasets and Benchmarks*.
- [46] Yao Tian, Ziyang Yue, Ruiyuan Zhang, Xi Zhao, Bolong Zheng, and Xiaofang Zhou. 2023. Approximate Nearest Neighbor Search in High Dimensional Vector Databases: Current Research and Future Directions. *IEEE Data Eng. Bull.* 47, 3 (2023), 39–54.
- [47] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2022. DB-LSH: Locality-Sensitive Hashing with Query-based Dynamic Bucketing. In *ICDE*. IEEE, 2250–2262.
- [48] Yao Tian, Xi Zhao, and Xiaofang Zhou. 2024. DB-LSH 2.0: Locality-Sensitive Hashing With Query-Based Dynamic Bucketing. *IEEE Trans. Knowl. Data Eng.* 36, 3 (2024), 1000–1015.
- [49] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xianguy Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1 (2024), V2mod014:1–V2mod014:27.
- [50] Xiao Wang, Craig Macdonald, Nicola Tonello, and Iadh Ounis. 2021. Pseudo-Relevance Feedback for Multiple Representation Dense Retrieval. In *ICTIR*. ACM, 297–306.

- [51] Xiao Wang, Craig Macdonald, Nicola Tonellotto, and Iadh Ounis. 2023. Reproducibility, Replicability, and Insights into Dense Multi-Representation Retrieval Models: from ColBERT to Col. In *SIGIR*. ACM, 2552–2561.
- [52] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, Lingkun Kong, Brian Moran, Jiaqi Wang, Yifan Ethan Xu, An Yan, Chenyu Yang, Eting Yuan, Hanwen Zha, Nan Tang, Lei Chen, Nicolas Scheffer, Yue Liu, Nirav Shah, Rakesh Wanga, Anuj Kumar, Wen-tau Yih, and Xin Luna Dong. 2024. CRAG - Comprehensive RAG Benchmark. *CoRR* abs/2406.04744 (2024).
- [53] Lewei Yao, Runhui Huang, Lu Hou, Guansong Lu, Minzhe Niu, Hang Xu, Xiaodan Liang, Zhenguo Li, Xin Jiang, and Chunjing Xu. 2022. FILIP: Fine-grained Interactive Language-Image Pre-Training. In *ICLR*. OpenReview.net.
- [54] Jingtao Zhan, Xiaohui Xie, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2022. Evaluating Interpolation and Extrapolation Performance of Neural Retrieval Models. In *CIKM*. ACM, 2486–2496.
- [55] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 16, 8 (2023), 1979–1991.
- [56] Bolong Zheng, Xi Zhao, Lianggui Weng, Nguyen Quoc Viet Hung, Hang Liu, and Christian S. Jensen. 2020. PM-LSH: A Fast and Accurate LSH Framework for High-Dimensional Approximate NN Search. *Proc. VLDB Endow.* 13, 5 (2020), 643–655.